

Stage 3 — Agent Memory with Postgres + pgvector

Goal: Give your agents real memory — short-term (per session), long-term (vector search), and episodic (run logs) — using **PostgreSQL + pgvector**, not local JSON or in-memory hacks. This is your first enterprise-grade memory layer.

What You Already Have (from Stage 1 & 2)

From Stage 1:

- A solid **single-agent loop** with tool calling.
- A **tool registry**.

From Stage 2:

- A **BaseAgent** with shared loop logic.
- **Specialist agents** (ResearchAgent, WriterAgent, ReviewerAgent).
- A **Code-Driven Orchestrator** and an **LLM-Driven Orchestrator**.

In all of that, memory is only in the **message list in RAM**. Once the process restarts, everything is forgotten. There is no cross-session memory, no long-term knowledge, no analytics.

Stage 3 fixes that by adding a **real memory stack**:

- **Short-term memory** (conversation-level) in Postgres.[web:174][web:176]
- **Long-term semantic memory** with pgvector (embeddings + similarity search).[web:177][web:181][web:186]
- **Episodic memory** (run logs) for analysis and replay.[web:173][web:183][web:184]

All of this is built on **Postgres + pgvector**, which in 2026 is a first-class vector DB option with competitive performance for most SaaS workloads.[web:181][web:185]

Concept 1 — Memory Types for Agents

Drawing from agent-memory guides and cognitive science-inspired designs, a practical agent memory system for your use case has three main types:[web:173][web:175][web:183]

1. Short-term memory

- **Scope:** One session / one campaign.
- **What:** All messages, tool calls, and intermediate outputs for that run.
- **Implementation:** Conversation table in Postgres, plus a summarization step to keep context compact.[web:174][web:176]

2. Long-term semantic memory (what you know)

- Scope: Across sessions / across campaigns.
- What: Brand guides, past approved content, past research insights, etc.
- Implementation: pgvector table with embeddings + metadata for semantic search.
[web:177][web:178][web:186]

3. Episodic memory (what happened)

- Scope: Every agent run.
- What: Timelines of agent actions, decisions, and outcomes — for debugging, analytics, and learning.
- Implementation: Run and event logs in Postgres with timestamps and types.[web:173]
[web:179][web:183]

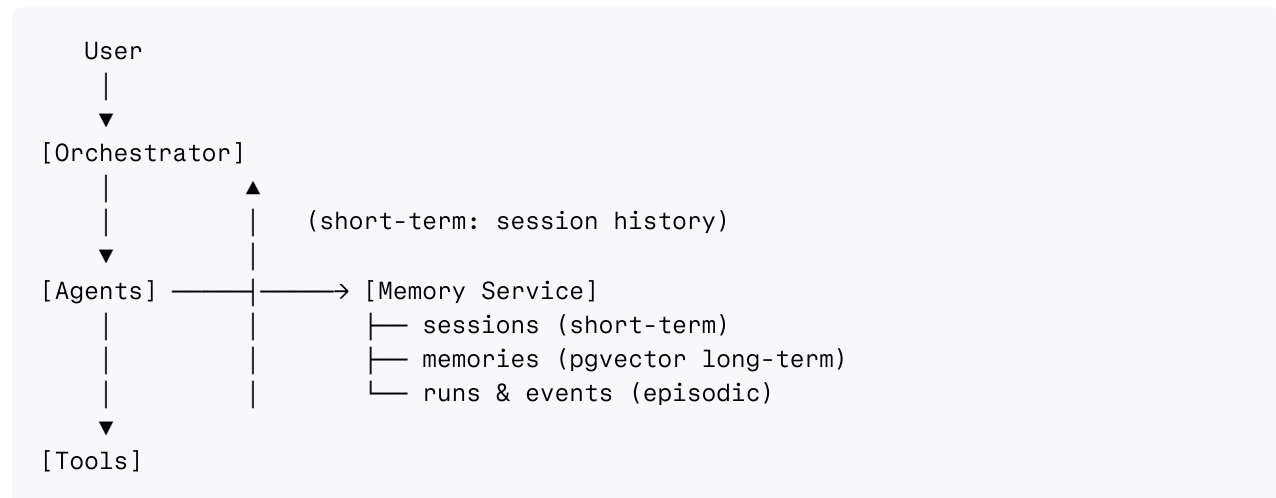
Your Creator Campaign Orchestrator will use **all three**:

- Short-term memory per campaign.
- Long-term memory for brand knowledge, past clips, etc.
- Episodic memory for auditing and improvement.

Stage 3 gives you an enterprise-ready skeleton of all three.

High-Level Architecture with Memory

Add a **Memory Layer** to the Stage 2 diagram:



The **Memory Service** is a Python module + Postgres schema that all agents and orchestrators call. No agent talks directly to Postgres tables; they talk to `memory.write_*` and `memory.read_*` functions.

Project Structure Additions

Starting from your Stage 2 repo, add a `memory/` package and SQL migrations:

```
stage3-memory/  
├── .env  
├── config.py  
├── requirements.txt  
├── migrations/  
│   └── 001_init_memory.sql  
├── memory/  
│   ├── __init__.py  
│   ├── db.py           ← central DB connection + helpers  
│   ├── models.py       ← typed dataclasses for memory objects  
│   ├── short_term.py   ← session & message storage  
│   ├── long_term.py    ← pgvector semantic memory (brand/knowledge)  
│   └── episodic.py      ← run and event logs  
├── agents/             ← from Stage 2 (with small changes)  
├── orchestrator/       ← from Stage 2 (with small changes)  
└── main.py
```

We will:

1. Provision Postgres with pgvector.
2. Create tables via migration.
3. Implement the Memory Service.
4. Wire agents and orchestrators to use it.

Step 1 — Postgres + pgvector Setup

1. Choose a managed Postgres with pgvector

For an enterprise-aligned setup, use a managed Postgres that supports pgvector out of the box or via `CREATE EXTENSION`:

- **Railway pgvector template** (Postgres + pgvector preconfigured).[web:178]
- **Timescale / pgEdge / Neon** with pgvector extension enabled.[web:177][web:182]

Avoid local SQLite or JSON files — you want a production-ready data store with:

- Backups.
- Indices.
- Connection pooling.
- Operational tooling.

2. Enable pgvector

Once you have a Postgres URL, connect (psql or GUI) and run:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

This adds the vector type and similarity operators Postgres needs for semantic search.
[web:177][web:181][web:186]

3. Add DB connection to .env

.env.example:

```
OPENAI_API_KEY=your_key_here
ORCHESTRATOR_MODEL=gpt-4o
AGENT_MODEL=gpt-4o-mini
DATABASE_URL=postgresql://user:pass@host:port/dbname
```

config.py (extend your existing one):

```
import os
from dotenv import load_dotenv

load_dotenv()

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
ORCHESTRATOR_MODEL = os.getenv("ORCHESTRATOR_MODEL", "gpt-4o")
AGENT_MODEL = os.getenv("AGENT_MODEL", "gpt-4o-mini")
DATABASE_URL = os.getenv("DATABASE_URL")

if not OPENAI_API_KEY:
    raise ValueError("OPENAI_API_KEY is not set in .env")
if not DATABASE_URL:
    raise ValueError("DATABASE_URL is not set in .env")
```

Why DATABASE_URL and not ad-hoc envs?

This is the same pattern used by production RAG pipelines using Postgres with pgvector — one URL keeps configuration simple and matches cloud provider dashboards.[web:178][web:182]

Step 2 — Database Schema (Migration)

migrations/001_init_memory.sql

```
-- Enable pgvector
CREATE EXTENSION IF NOT EXISTS vector;

-- 1. Sessions (short-term memory scope)
CREATE TABLE IF NOT EXISTS sessions (
```

```

    id            UUID PRIMARY KEY,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    user_id       TEXT,                -- optional: multi-tenant support
    topic         TEXT,                -- e.g., blog topic or campaign name
    metadata      JSONB DEFAULT '{}'::jsonb
);

-- 2. Messages (short-term memory content)
CREATE TABLE IF NOT EXISTS messages (
    id            BIGSERIAL PRIMARY KEY,
    session_id    UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    role         TEXT NOT NULL,        -- 'user', 'assistant', 'tool', 'system'
    content       TEXT NOT NULL,
    metadata      JSONB DEFAULT '{}'::jsonb
);

CREATE INDEX IF NOT EXISTS idx_messages_session_created
    ON messages (session_id, created_at);

-- 3. Long-term semantic memory (pgvector)
-- Store facts, brand rules, approved content snippets, etc.

-- Adjust embedding dimension (1536 for OpenAI text-embedding-3-large)
CREATE TABLE IF NOT EXISTS memories (
    id            BIGSERIAL PRIMARY KEY,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    session_id    UUID,                -- optional link to session
    user_id       TEXT,                -- optional multi-tenant
    kind          TEXT NOT NULL,        -- 'brand', 'fact', 'clip', 'insight', etc.
    label         TEXT NOT NULL,        -- short description
    content       TEXT NOT NULL,        -- full text
    metadata      JSONB DEFAULT '{}'::jsonb,
    embedding     vector(1536)         -- pgvector column
);

CREATE INDEX IF NOT EXISTS idx_memories_kind ON memories(kind);
CREATE INDEX IF NOT EXISTS idx_memories_user ON memories(user_id);

-- HNSW index for fast similarity search (cosine distance)
CREATE INDEX IF NOT EXISTS idx_memories_embedding
    ON memories
    USING hnsw (embedding vector_cosine_ops);

-- 4. Episodic memory: runs and events

CREATE TABLE IF NOT EXISTS runs (
    id            UUID PRIMARY KEY,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    session_id    UUID REFERENCES sessions(id) ON DELETE SET NULL,
    orchestrator  TEXT NOT NULL,        -- 'code-driven', 'llm-driven', etc.
    status        TEXT NOT NULL,        -- 'success', 'error'
    error         TEXT,                -- error message if any
    metadata      JSONB DEFAULT '{}'::jsonb
);

```

```
CREATE TABLE IF NOT EXISTS events (
    id            BIGSERIAL PRIMARY KEY,
    run_id        UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    agent_name    TEXT,                -- e.g. 'ResearchAgent'
    event_type    TEXT NOT NULL,       -- 'agent_start', 'agent_end', 'tool_call', etc.
    message       TEXT,                -- human-readable description
    payload       JSONB DEFAULT '{}'::jsonb
);

CREATE INDEX IF NOT EXISTS idx_events_run_created
ON events (run_id, created_at);
```

Why this schema?

- sessions + messages = short-term memory layer.[web:174][web:176]
- memories with vector column = long-term semantic memory (pgvector).[web:177][web:181][web:186]
- runs + events = episodic memory, enabling replay and analytics.[web:173][web:183]

Apply the migration once:

```
psql "$DATABASE_URL" -f migrations/001_init_memory.sql
```

Use your actual connection string, or a GUI if you prefer.

Step 3 — Memory DB Helper

requirements.txt additions

```
psycopg[binary]>=3.1.0
openai>=1.30.0
python-dotenv>=1.0.0
pydantic>=2.0.0
```

memory/db.py

```
import psycopg
from contextlib import contextmanager
from config import DATABASE_URL

@contextmanager
def get_conn():
    conn = psycopg.connect(DATABASE_URL)
    try:
        yield conn
    finally:
```

```

        conn.close()

@contextmanager
def get_cursor():
    with get_conn() as conn:
        with conn.cursor() as cur:
            yield cur
            conn.commit()

```

This thin layer gives you consistent connection/commit logic, invertible for future pooling libraries.

Step 4 — Typed Models for Memory

memory/models.py

```

from dataclasses import dataclass
from datetime import datetime
from typing import Any, Dict, List, Optional

@dataclass
class Session:
    id: str
    created_at: datetime
    updated_at: datetime
    user_id: Optional[str]
    topic: Optional[str]
    metadata: Dict[str, Any]

@dataclass
class Message:
    id: int
    session_id: str
    created_at: datetime
    role: str
    content: str
    metadata: Dict[str, Any]

@dataclass
class MemoryRecord:
    id: int
    created_at: datetime
    session_id: Optional[str]
    user_id: Optional[str]
    kind: str
    label: str
    content: str
    metadata: Dict[str, Any]
    embedding: List[float]

```

```

@dataclass
class Run:
    id: str
    created_at: datetime
    session_id: Optional[str]
    orchestrator: str
    status: str
    error: Optional[str]
    metadata: Dict[str, Any]

```

```

@dataclass
class Event:
    id: int
    run_id: str
    created_at: datetime
    agent_name: Optional[str]
    event_type: str
    message: str
    payload: Dict[str, Any]

```

These dataclasses make it easier to reason about what you're storing and returning.

Step 5 — Short-Term Memory (Sessions + Messages)

memory/short_term.py

```

import uuid
from typing import Optional, List, Dict, Any
from datetime import datetime
from memory.db import get_cursor
from memory.models import Session, Message

def create_session(user_id: Optional[str] = None,
                  topic: Optional[str] = None,
                  metadata: Optional[Dict[str, Any]] = None) -> str:
    """Create a new session and return its UUID string."""
    session_id = str(uuid.uuid4())
    metadata = metadata or {}
    with get_cursor() as cur:
        cur.execute(
            """
            INSERT INTO sessions (id, user_id, topic, metadata)
            VALUES (%s, %s, %s, %s)
            """,
            (session_id, user_id, topic, metadata),
        )
    return session_id

```



```

def append_message(session_id: str, role: str, content: str,
                   metadata: Optional[Dict[str, Any]] = None) -> None:
    """Append a message to the short-term memory for this session."""
    metadata = metadata or {}
    with get_cursor() as cur:
        cur.execute(
            """
            INSERT INTO messages (session_id, role, content, metadata)
            VALUES (%s, %s, %s, %s)
            """,
            (session_id, role, content, metadata),
        )
        cur.execute(
            "UPDATE sessions SET updated_at = now() WHERE id = %s",
            (session_id,),
        )

def get_recent_messages(session_id: str, limit: int = 20) -> List[Message]:
    """Fetch recent messages for a session, ordered by created_at."""
    with get_cursor() as cur:
        cur.execute(
            """
            SELECT id, session_id, created_at, role, content, metadata
            FROM messages
            WHERE session_id = %s
            ORDER BY created_at DESC
            LIMIT %s
            """,
            (session_id, limit),
        )
        rows = cur.fetchall()
    messages = [
        Message(
            id=row[0],
            session_id=row[1],
            created_at=row[2],
            role=row[3],
            content=row[4],
            metadata=row[5] or {},
        )
        for row in reversed(rows) # reverse to oldest→newest
    ]
    return messages

def summarize_history_text(messages: List[Message]) -> str:
    """Simple concatenation for now; will replace with LLM summarization later."""
    parts = []
    for m in messages:
        prefix = "User" if m.role == "user" else "Assistant" if m.role == "assistant"
        parts.append(f"[{prefix}] {m.content}")
    return "\n".join(parts)

```

Notes:

- `create_session()` returns a UUID string — this will be passed into `BaseAgent.run()` soon.
- `append_message()` should be called for every message you send to / receive from the model.
- `get_recent_messages()` lets you reconstruct the last N messages and pass them into the LLM as context when continuing a session.

A more advanced version would use LLM summarization to compress history, as recommended by modern agent memory guidance — but simple concatenation is enough to understand the pattern first.[web:161][web:180]

Step 6 — Long-Term Memory (pgvector)

Long-term memory stores facts, brand rules, and reusable knowledge as embeddings in Postgres.[web:173][web:180][web:186]

memory/long_term.py

```
from typing import List, Dict, Any, Optional, Tuple
from openai import OpenAI
from config import OPENAI_API_KEY
from memory.db import get_cursor
from memory.models import MemoryRecord

client = OpenAI(api_key=OPENAI_API_KEY)
EMBEDDING_MODEL = "text-embedding-3-large" # 1536-dim

def _embed(text: str) -> List[float]:
    response = client.embeddings.create(
        model=EMBEDDING_MODEL,
        input=text,
    )
    return response.data[0].embedding

def add_memory(kind: str,
               label: str,
               content: str,
               session_id: Optional[str] = None,
               user_id: Optional[str] = None,
               metadata: Optional[Dict[str, Any]] = None) -> int:
    """Insert a new long-term memory and return its ID."""
    metadata = metadata or {}
    embedding = _embed(content)
    with get_cursor() as cur:
        cur.execute(
            """
            INSERT INTO memories (session_id, user_id, kind, label, content, metadata, embedding)
            VALUES (%s, %s, %s, %s, %s, %s, %s)
            RETURNING id
            """,
            (session_id, user_id, kind, label, content, metadata, embedding),
        )
```

```

        mem_id = cur.fetchone()[0]
    return mem_id

def search_memory(query: str,
                  user_id: Optional[str] = None,
                  kind: Optional[str] = None,
                  limit: int = 5) -> List[Tuple[MemoryRecord, float]]:
    """Semantic search over memories using cosine distance (lower is closer)."""
    query_embedding = _embed(query)
    with get_cursor() as cur:
        base = (
            "SELECT id, created_at, session_id, user_id, kind, label, content, metadata, "
            "embedding <-> %s AS distance "
            "FROM memories"
        )
        params: List[Any] = [query_embedding]
        conditions: List[str] = []

        if user_id:
            conditions.append("user_id = %s")
            params.append(user_id)
        if kind:
            conditions.append("kind = %s")
            params.append(kind)

        if conditions:
            base += " WHERE " + " AND ".join(conditions)
        base += " ORDER BY distance ASC LIMIT %s"
        params.append(limit)

        cur.execute(base, params)
        rows = cur.fetchall()

    results: List[Tuple[MemoryRecord, float]] = []
    for row in rows:
        record = MemoryRecord(
            id=row[0],
            created_at=row[1],
            session_id=row[2],
            user_id=row[3],
            kind=row[4],
            label=row[5],
            content=row[6],
            metadata=row[7] or {},
            embedding=list(row[8]),
        )
        distance = float(row[9])
        results.append((record, distance))
    return results

```

How this maps to your Creator Orchestrator later:

- kind="brand" for brand guide items.
- kind="clip" for past approved clips.

- `kind="insight"` for learned lessons about what performs well.

Before `PlanningAgent` proposes new clips, it can `search_memory("hooks that performed well for B2B AI")` and get the top 5 references.[\[web:173\]](#)[\[web:183\]](#)

Step 7 — Episodic Memory (Runs + Events)

Episodic memory tracks **what happened in each run** so you can debug, replay, and analyze.
[\[web:173\]](#)[\[web:179\]](#)[\[web:183\]](#)

memory/episodic.py

```
import uuid
from typing import Optional, Dict, Any
from memory.db import get_cursor

def start_run(session_id: Optional[str], orchestrator: str,
              metadata: Optional[Dict[str, Any]] = None) -> str:
    """Create a run record and return its UUID."""
    run_id = str(uuid.uuid4())
    metadata = metadata or {}
    with get_cursor() as cur:
        cur.execute(
            """
            INSERT INTO runs (id, session_id, orchestrator, status, metadata)
            VALUES (%s, %s, %s, %s, %s)
            """,
            (run_id, session_id, orchestrator, "running", metadata),
        )
    return run_id

def finish_run(run_id: str, status: str = "success", error: Optional[str] = None) ->
    with get_cursor() as cur:
        cur.execute(
            """
            UPDATE runs
            SET status = %s, error = %s
            WHERE id = %s
            """,
            (status, error, run_id),
        )

def log_event(run_id: str,
              event_type: str,
              message: str,
              agent_name: Optional[str] = None,
              payload: Optional[Dict[str, Any]] = None) -> None:
    payload = payload or {}
    with get_cursor() as cur:
        cur.execute(
            """
```

```

        INSERT INTO events (run_id, agent_name, event_type, message, payload)
        VALUES (%s, %s, %s, %s, %s)
        """
        (run_id, agent_name, event_type, message, payload),
    )

```

Examples of events to log:

- `agent_start` / `agent_end` with `agent_name` and `topic`.
- `tool_call` with `payload` containing tool name + arguments.
- `memory_write` when adding long-term memories.

Later, you can build a simple UI to replay what a run did — similar to what LangGraph Studio offers for graph debugging.[web:83]

Step 8 — Wiring Memory into BaseAgent

Now we update `BaseAgent` so every agent run knows about `session_id` and logs to episodic memory.

agents/base.py (updated)

```

import json
from typing import Optional
from openai import OpenAI
from config import OPENAI_API_KEY
from tools.registry import get_schemas, execute
from memory import short_term, episodic

class BaseAgent:
    name: str = "BaseAgent"
    system_prompt: str = "You are a helpful assistant."
    allowed_tools: list[str] = []
    model: str = "gpt-4o-mini"
    max_iterations: int = 6

    def __init__(self):
        self.client = OpenAI(api_key=OPENAI_API_KEY)

    def run(self,
            user_message: str,
            context: str = "",
            session_id: Optional[str] = None,
            run_id: Optional[str] = None) -> str:
        """Run this agent, optionally attached to a session and run."""
        # If no session provided, create one
        if session_id is None:
            session_id = short_term.create_session(topic=self.name)

        # Log agent start
        if run_id is not None:

```

```

        episodic.log_event(
            run_id,
            event_type="agent_start",
            message=f"{self.name} starting",
            agent_name=self.name,
            payload={"session_id": session_id},
        )

system = self.system_prompt
if context:
    system += f"\n\n--- CONTEXT PROVIDED ---\n{context}"

# Optional: include recent history in short-term memory
history_messages = short_term.get_recent_messages(session_id, limit=10)
history_text = short_term.summarize_history_text(history_messages)
if history_text:
    system += f"\n\n--- RECENT HISTORY ---\n{history_text}"

messages = [
    {"role": "system", "content": system},
    {"role": "user", "content": user_message},
]

# Persist messages
short_term.append_message(session_id, "system", system)
short_term.append_message(session_id, "user", user_message)

tools = get_schemas(self.allowed_tools) if self.allowed_tools else []
call_kwargs = {"model": self.model, "messages": messages}
if tools:
    call_kwargs["tools"] = tools
    call_kwargs["tool_choice"] = "auto"

for iteration in range(self.max_iterations):
    response = self.client.chat.completions.create(**call_kwargs)
    message = response.choices[0].message

    if message.tool_calls:
        messages.append(message)
        short_term.append_message(session_id, "assistant", json.dumps({"tool.

        for tool_call in message.tool_calls:
            tool_name = tool_call.function.name
            arguments = json.loads(tool_call.function.arguments)

            # Log tool call
            if run_id is not None:
                episodic.log_event(
                    run_id,
                    event_type="tool_call",
                    message=f"{self.name} calling {tool_name}",
                    agent_name=self.name,
                    payload={"arguments": arguments},
                )

            result = execute(tool_name, arguments)

```

```

        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": result,
        })
        short_term.append_message(session_id, "tool", result, {"tool": tool_call.id})

    call_kwargs["messages"] = messages
else:
    final = message.content or ""
    short_term.append_message(session_id, "assistant", final)

    if run_id is not None:
        episodic.log_event(
            run_id,
            event_type="agent_end",
            message=f"{self.name} finished",
            agent_name=self.name,
            payload={"final_preview": final[:200]},
        )

    return final

if run_id is not None:
    episodic.log_event(
        run_id,
        event_type="agent_error",
        message=f"{self.name} exceeded max iterations",
        agent_name=self.name,
        payload={},
    )
return "Error: agent loop exceeded max iterations."

```

Key changes:

- `session_id` is now a first-class parameter — every agent call can be tied into a session.
- Short-term memory is read (`get_recent_messages`) to prime the system prompt with recent history.
- Every message (system, user, tool, assistant) is appended to the messages table.
- Episodic logging captures agent start/end and tool calls per `run_id`.

Step 9 — Wiring Memory into the Orchestrator

Code-Driven Orchestrator (example)

Update `orchestrator/code_driven.py` to:

- Create a `session_id` at the start.
- Create a `run_id` with `orchestrator="code-driven"`.
- Pass both into each agent.

- Finish the run at the end.

```

from memory import short_term, episodic

class CodeDrivenOrchestrator:
    # ... existing __init__ ...

    def run(self, topic: str, user_id: str | None = None) -> dict:
        print(f"\n{'='*60}")
        print(f"CODE-DRIVEN ORCHESTRATOR: '{topic}'")
        print(f"{'='*60}")

        session_id = short_term.create_session(user_id=user_id, topic=topic)
        run_id      = episodic.start_run(session_id, orchestrator="code-driven", meta

        try:
            # Step 1: Research
            research_text = self.research_agent.run(
                f"Research this topic: {topic}",
                session_id=session_id,
                run_id=run_id,
            )
            # (Then structured parsing as before...)
            # Step 2, Step 3 unchanged except they pass session_id & run_id

            result = {
                "topic": topic,
                # include research/draft/review as before...
                "session_id": session_id,
                "run_id": run_id,
                "pipeline": "code-driven",
            }
            episodic.finish_run(run_id, status="success")
            return result

        except Exception as e:
            episodic.finish_run(run_id, status="error", error=str(e))
            raise

```

The LLM-driven orchestrator should do the same — create `session_id` + `run_id`, pass them into each agent wrapper, and log events.

Step 10 — Using Long-Term Memory in an Agent

As a concrete example, extend `ResearchAgent` to **write important facts to long-term memory**.

agents/research_agent.py (sketch)

```

from agents.base import BaseAgent
from config import AGENT_MODEL
from memory import long_term, episodic

```



```

class ResearchAgent(BaseAgent):
    name = "ResearchAgent"
    model = AGENT_MODEL
    allowed_tools = ["web_search"]
    system_prompt = """You are a specialist research agent.

Your job is to research a topic and produce:
- A concise summary.
- 3-5 key points.
- Important facts or statistics.
After you respond, the orchestrator may store your key facts in long-term memory.
"""

    def run_and_store(self, topic: str, session_id: str, run_id: str | None = None):
        """Run normal research, then extract & store facts in long-term memory."""
        base_response = super().run(
            user_message=f"Research this topic: {topic}",
            session_id=session_id,
            run_id=run_id,
        )

        # Simple heuristic: treat bullet lines starting with '-' as facts
        facts = [
            line[1:].strip()
            for line in base_response.splitlines()
            if line.strip().startswith("-")
        ]

        for fact in facts:
            mem_id = long_term.add_memory(
                kind="insight",
                label=f"Fact about {topic}",
                content=fact,
                session_id=session_id,
            )
            if run_id:
                episodic.log_event(
                    run_id,
                    event_type="memory_write",
                    message=f"Stored fact as memory {mem_id}",
                    agent_name=self.name,
                    payload={"fact": fact},
                )

        return base_response

```

Later, a PlanningAgent in your Creator Orchestrator can call `search_memory()` to retrieve these facts and use them in new campaigns.

How to Test Stage 3 End-to-End

1. Run migration and verify tables exist.
2. Run orchestrator on a topic:

```
python main.py
```

Choose code-driven pipeline, provide a topic.

3. After the run, connect to Postgres and inspect:
 - `SELECT * FROM sessions ORDER BY created_at DESC LIMIT 3;`
 - `SELECT * FROM messages WHERE session_id = '...' ORDER BY created_at;`
 - `SELECT * FROM runs ORDER BY created_at DESC LIMIT 3;`
 - `SELECT * FROM events WHERE run_id = '...' ORDER BY created_at;`
4. Call `search_memory("content creation", kind='insight')` from a REPL and confirm it retrieves stored facts.

You should see:

- A new session with topic = your blog topic.
- Messages capturing each agent's conversation.
- A run entry with status success.
- Events for agent_start, tool_call, agent_end, maybe memory_write.
- Memories table populated (if you implemented `run_and_store`).

Enterprise Alignment Check — Are We Still "Toy"?

At Stage 3, your architecture is no longer "local toy":

- **Postgres + pgvector** as the unified data store (sessions, memories, runs).[\[web:177\]](#)[\[web:181\]](#)[\[web:186\]](#)
- **Clear separation:** app logic vs. memory layer vs. DB.[\[web:178\]](#)[\[web:182\]](#)
- **Memory types** aligned with best practice: short-term vs. long-term vs. episodic.[\[web:173\]](#)[\[web:180\]](#)[\[web:183\]](#)
- **Per-agent memory access** through a service, not direct SQL from random places.

This is exactly the pattern recommended by modern RAG and agent-memory guides: unify structured data and vectors in Postgres instead of gluing a separate vector DB until you truly need it.[\[web:181\]](#)[\[web:185\]](#)[\[web:186\]](#)

Completion Criteria for Stage 3

You can mark Stage 3 as "done" when:

- ☐ You can run the orchestrator, then query PostgreSQL and see:
 - A sessions row with your topic.
 - messages rows showing the conversation.
 - A runs row with status success.
 - events rows for agent_start, tool_call, agent_end.
- ☐ You can manually call `search_memory()` and get meaningful results.
- ☐ You can explain to someone else the difference between:
 - Short-term memory in messages.
 - Long-term memory in memories.
 - Episodic memory in runs + events.
- ☐ You understand why Postgres + pgvector is chosen over local JSON files or an ad-hoc in-memory store.

Once these are true, Stage 4 (Guardrails) will tie directly into this: guardrails will use your memory (brand rules, history) to decide when to block, allow, or escalate actions — exactly what your Creator Campaign Orchestrator needs.